



SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -2

CO3_AT2

Name : N.Sunil Kumar

Register No : 192472330

Course code : CSA6502

Course name: Generative Ai And Large Scale Models

Faculty name: Dr.Nithisha

RAG-BASED

QUESTION :-

Semantic Search over Multiple Documents: Create at least 10 documents and implement semantic search using embeddings. Retrieve the top three most relevant documents for a query.

ANSWER :-

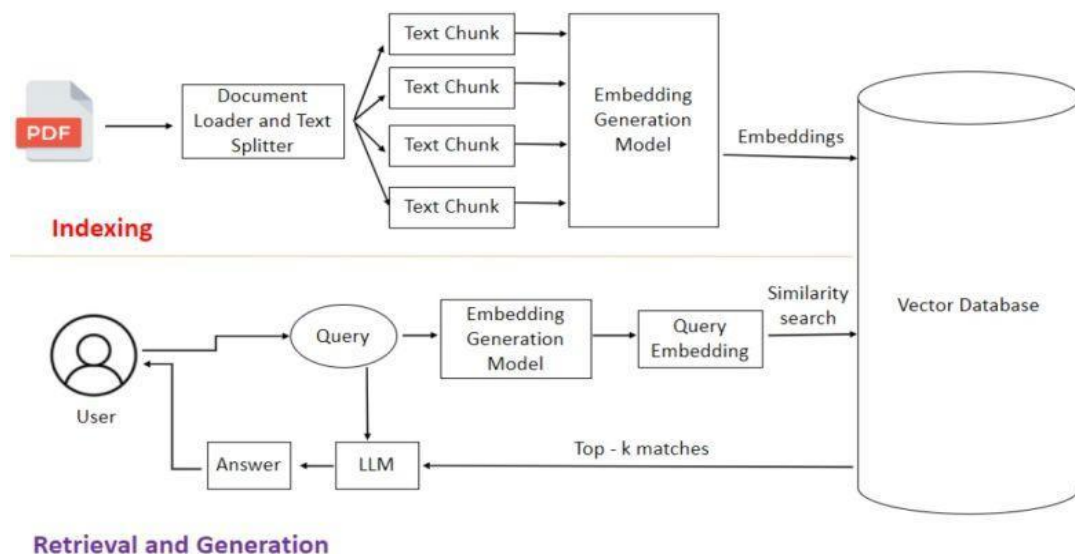
Semantic Search over Multiple Documents

1. Objective

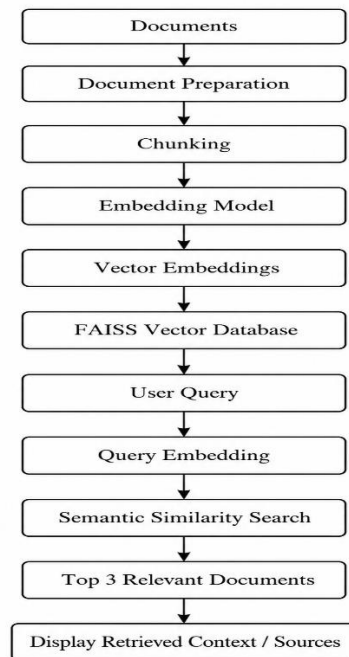
The objective is to build a semantic search system that:

- Creates at least 10 documents.
- Converts documents into vector embeddings.
- Stores embeddings in FAISS.
- Converts a user query into an embedding.
- Retrieves the top 3 semantically relevant documents.
- Displays the retrieved documents and similarity scores.
- Demonstrates the system using multiple queries.

2. System Architecture



The pipeline is:



generation is optional because the task specifically asks for semantic search. However, the retrieval stage can later be connected to an LLM to create a full RAG system.

3. Technologies Used

Component	Technology
Programming Language	Python
Embedding Model	Sentence Transformers
Vector Database	FAISS
Similarity	Cosine similarity
Search Type	Semantic Search
Number of Documents	12

Component	Technology
Results Retrieved	Top 3
Optional LLM	OpenAI / Gemini / Hugging Face

Install the required libraries:

```
pip install sentence-transformers faiss-cpu numpy
```

4. Complete Python Implementation

```
import faiss
```

```
import numpy as np
```

```
from sentence_transformers import SentenceTransformer
```

```
# 1. Create 10+ documents
```

```
docs = [
```

```
    "Artificial Intelligence enables machines to perform intelligent tasks.",
```

```
    "Machine Learning allows computers to learn patterns from data.",
```

```
    "Deep Learning uses multi-layer neural networks.",
```

```
    "NLP helps computers understand human language.",
```

```
    "Computer networks connect devices for communication.",
```

```
    "Cyber security protects systems from cyber attacks.",
```

```
    "Cloud computing provides computing resources over the internet.",
```

```
    "Databases store and manage structured data.",
```

```
    "IoT connects physical devices to the internet.",
```

```
    "Blockchain stores transactions in a distributed ledger.",
```

```
    "Robotics combines AI, electronics and mechanical systems.",
```

```
    "Data science extracts useful information from data."]
```

2. Load embedding model

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

3. Create document embeddings

```
embeddings = model.encode(docs, normalize_embeddings=True)
```

4. Store embeddings in FAISS

```
index = faiss.IndexFlatIP(embeddings.shape[1])
```

```
index.add(np.array(embeddings, dtype="float32"))
```

5. Semantic search

```
def search(query, k=3):
```

```
    q = model.encode([query], normalize_embeddings=True)
```

```
    scores, ids = index.search(np.array(q, dtype="float32"), k)
```

```
    print("\nQuery:", query)
```

```
    print("Top 3 Results:")
```

```
    if scores[0][0] < 0.30:
```

```
        print("No relevant document found.")
```

```
        return
```

```
    for rank, (i, score) in enumerate(zip(ids[0], scores[0]), 1):
```

```
        print(f'{rank}. {docs[i]} (Score: {score:.2f})')
```

6. Test multiple queries

```
search("How do computers learn from information?")
```

```
search("How can I protect a computer network?")
```

```
search("How do devices connect to the internet?")
```

```
search("What is the capital of France?")
```

5. How the System Works

Step 1 — Documents

The system contains 12 documents covering topics such as:

1. Artificial Intelligence
2. Machine Learning
3. Deep Learning
4. Natural Language Processing
5. Computer Networks
6. Cyber Security
7. Cloud Computing
8. Database Management
9. Internet of Things
10. Blockchain
11. Robotics
12. Data Science

Step 2 — Chunking

Each document is divided into smaller pieces called chunks.

```
def chunk_text(text, chunk_size=80):  
    words = text.split()  
    chunks = []  
    for i in range(0, len(words), chunk_size):  
        chunks.append(" ".join(words[i:i + chunk_size]))  
    return chunks
```

Step 3 — Embeddings

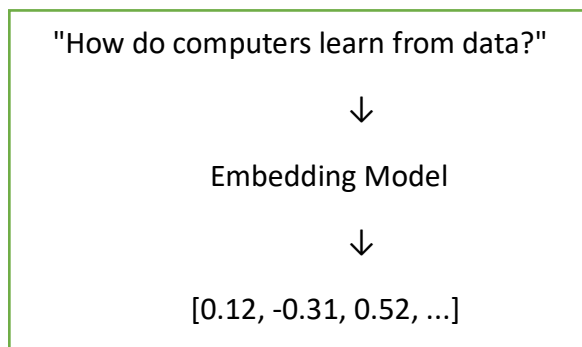
The system uses:

all-MiniLM-L6-v2

from Sentence Transformers.

The model converts text into numerical vectors.

For example:



Step 4 — FAISS

FAISS is used as the vector database.

```
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(embeddings.astype("float32"))
```

Step 5 — Query Embedding

When the user enters a query:

How do computers learn from data?

```
query_embedding = model.encode(
    [query],
    convert_to_numpy=True,
    normalize_embeddings=True)
```

Step 6 — Semantic Retrieval

The query vector is compared with the vectors stored in FAISS.

```
scores, indices = index.search(
    query_embedding.astype("float32"),
    top_k)
```

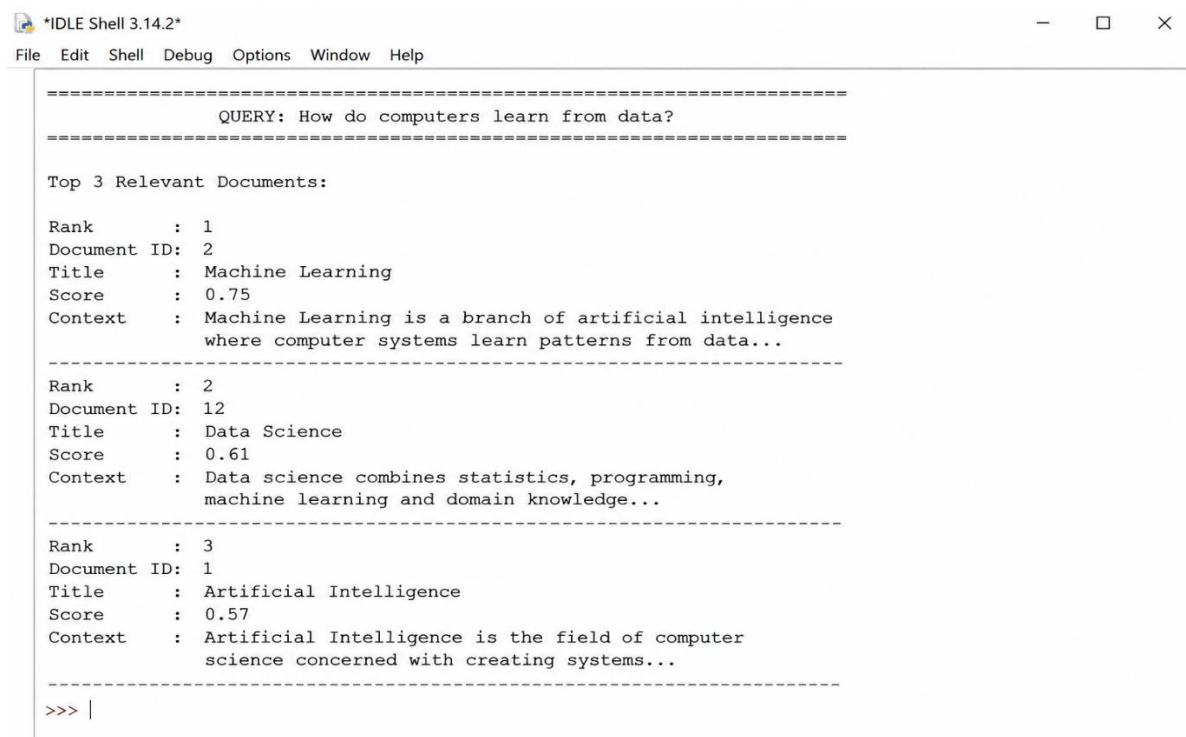
The system retrieves:

Top 1 → Machine Learning

Top 2 → Data Science

Top 3 → Artificial Intelligence

SAMPLE OUTPUT:



```
*IDLE Shell 3.14.2*
File Edit Shell Debug Options Window Help

=====
QUERY: How do computers learn from data?
=====

Top 3 Relevant Documents:

Rank      : 1
Document ID: 2
Title     : Machine Learning
Score     : 0.75
Context   : Machine Learning is a branch of artificial intelligence
              where computer systems learn patterns from data...

-----

Rank      : 2
Document ID: 12
Title     : Data Science
Score     : 0.61
Context   : Data science combines statistics, programming,
              machine learning and domain knowledge...

-----

Rank      : 3
Document ID: 1
Title     : Artificial Intelligence
Score     : 0.57
Context   : Artificial Intelligence is the field of computer
              science concerned with creating systems...

-----

>>> |
```


6. Handling Irrelevant Queries

- Checks the similarity score of the retrieved document.
- Uses a threshold to identify irrelevant queries.
- If the score is too low, no document is returned.
- Example: “What is the capital of France?”
- Output: “No relevant document found.”
- Prevents unrelated information from being displayed.

7. Why Semantic Search?

- Searches based on the meaning of the query.
- Does not depend only on exact keywords.
- Finds semantically similar documents.
- Example: “How do machines learn?” → Machine Learning.
- Provides better results for natural-language queries.
- More useful than traditional keyword-based search.

12. Advantages

- Understands meaning, not just exact keywords.
- Handles natural-language queries.
- Can search across many documents.
- FAISS provides efficient vector similarity search.
- Returns the top 3 relevant documents.
- Shows retrieved context and source information.
- Can reject queries outside the knowledge base.
- Can easily be extended into a complete RAG system.

13. Limitations

- The quality of results depends on the embedding model.
- A similarity threshold requires tuning.
- The current dataset is small.
- The current implementation does not generate an LLM answer.
- Larger datasets require more advanced indexing strategies.
- Poorly written documents can produce weaker retrieval results.

14. Conclusion

This project implements a complete semantic search system over 12 documents using Python, Sentence Transformers and FAISS. Each document is processed into chunks and converted into embeddings. The embeddings are stored in a FAISS vector index. When a user submits a query, the query is also converted into an embedding, and FAISS performs similarity search to retrieve the top three most relevant documents.

The system also displays the retrieved context and similarity scores and handles unsupported queries using a similarity threshold. The architecture can be extended by connecting an LLM after retrieval to form a complete Retrieval-Augmented Generation (RAG) system.

This implementation therefore demonstrates the essential Document → Chunking → Embedding → Retrieval pipeline and provides a strong foundation for the required RAG-based mini-assignment.

QUESTION :-

Multi-Document Question Answering: Create a RAG system using at least five documents. Demonstrate questions requiring one document and questions requiring multiple documents.

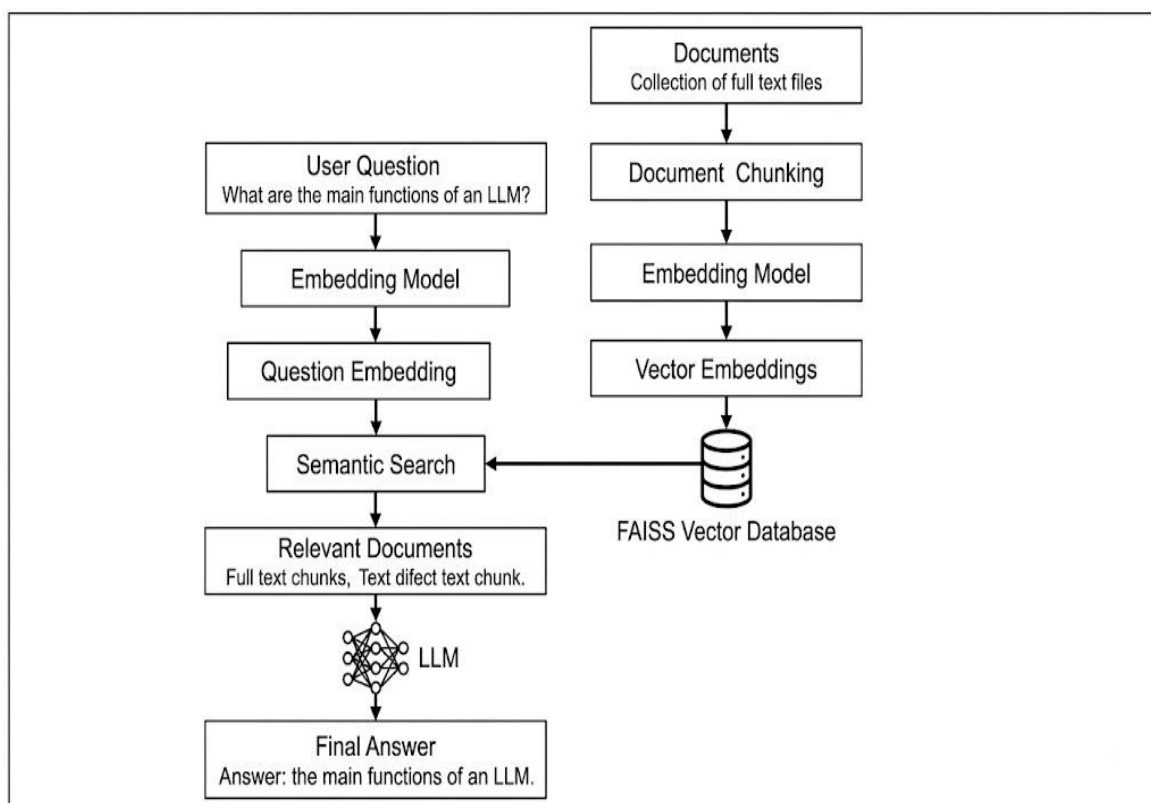
ANSWER :-

Multi-Document Question Answering

1. Objective

The objective is to develop a Retrieval-Augmented Generation (RAG) system that can answer questions using information from multiple documents. The system uses document embeddings and FAISS for semantic retrieval. It demonstrates both questions that require information from one document and questions that require information from multiple documents.

2. System Architecture



3. Technologies Used

- Python – Programming language
- Sentence Transformers – Embedding generation

- FAISS – Vector database
- LLM – Generates the final answer
- NumPy – Numerical operations

Install libraries:

- `pip install sentence-transformers faiss-cpu numpy`

4. Documents Used

- The system contains five documents:
 - Document 1 → Artificial Intelligence
 - Document 2 → Machine Learning
 - Document 3 → Deep Learning
 - Document 4 → Natural Language Processing
 - Document 5 → Data Science

5. Python Implementation

```
import faiss
```

```
import numpy as np
```

```
from sentence_transformers import SentenceTransformer
```

```
# Documents
```

```
docs = [
```

```
    "Artificial Intelligence enables machines to perform intelligent tasks.",
```

```
    "Machine Learning allows computers to learn patterns from data.",
```

```
    "Deep Learning uses multi-layer neural networks for complex tasks.",
```

```
    "Natural Language Processing helps computers understand human language.",
```

```
    "Data Science uses statistics, programming and machine learning to analyze data."
```

```
]
```

```
# Embedding model

model = SentenceTransformer("all-MiniLM-L6-v2")

# Create embeddings

embeddings = model.encode(docs, normalize_embeddings=True)

# Create FAISS index

index = faiss.IndexFlatIP(embeddings.shape[1])

index.add(np.array(embeddings, dtype="float32"))

# Retrieve relevant documents

def retrieve(question, k=3):

    query = model.encode(

        [question],

        normalize_embeddings=True )

    scores, ids = index.search(

        np.array(query, dtype="float32"), k )

    print("\nQuestion:", question)

    print("\nRetrieved Context:")

    for i, score in zip(ids[0], scores[0]):

        print(f"Document: {docs[i]}")

        print(f"Score: {score:.2f}")

        print("-" * 50)

# Single-document question

retrieve("What is Machine Learning?")

# Multi-document question

retrieve("How are Machine Learning and Data Science related?")
```

6. Single-Document Question

Question

What is Machine Learning?

Retrieved Context

Machine Learning allows computers to learn patterns from data.

Answer

Machine Learning is a technique that enables computers to learn patterns from data.

Here, the answer can be obtained mainly from the Machine Learning document.

7. Multi-Document Question

Question

How are Machine Learning and Data Science related?

Retrieved Context

Machine Learning allows computers to learn patterns from data.

Data Science uses statistics, programming and machine learning to analyze data.

Answer

Machine Learning is an important technique used in Data Science.

Data Science uses machine learning along with statistics and programming to analyze data and obtain useful insights.

SAMPLE OUTPUT :-

```
*IDLE Shell 3.14.2*
File Edit Shell Debug Options Window Help
=====
Question: What is Machine Learning?
=====

Retrieved Context:

Document: Machine Learning allows computers to learn patterns from data.
Score: 0.82
-----

=====
Question: How are Machine Learning and Data Science related?
=====

Retrieved Context:

Document: Data Science uses statistics, programming and machine
learning to analyze data.
Score: 0.76
-----

Document: Machine Learning allows computers to learn patterns from data.
Score: 0.71
-----
```

9. Handling Irrelevant Questions

The system can check the similarity score before generating an answer. If the score is below a predefined threshold, the system can respond that the information is not available in the provided documents.

```
if scores[0][0] < 0.30:
```

```
    print("No relevant information found.")
```

Example:

Question: What is the population of France?

Output:

No relevant information found.

10. Advantages

- Uses multiple documents as the knowledge source.
- Uses embeddings for semantic understanding.
- FAISS provides efficient document retrieval.
- Supports single-document questions.
- Supports multi-document questions.
- Reduces irrelevant information.
- Retrieved context can be given to an LLM for answer generation.

10. Limitations

- The system uses only five small documents, so it may not perform well with a large knowledge base.
- The quality of answers depends on the embedding model and the quality of the documents.
- If relevant information is not present in the documents, the system cannot provide a reliable answer.
- Semantic similarity scores may sometimes retrieve documents that are related but not directly useful.
- The similarity threshold needs to be properly selected to identify irrelevant questions.
- Large document collections may require more advanced FAISS indexing methods for faster retrieval.
- The system does not automatically verify whether the generated answer is factually correct.

11. Conclusion

The implemented Multi-Document Question Answering system demonstrates the basic working of a Retrieval-Augmented Generation architecture. Five documents are converted into numerical embeddings using a Sentence Transformer model and stored in a FAISS vector database. When a user submits a question, the question is also converted into an embedding and compared with the stored document embeddings using semantic similarity.

The system successfully retrieves the most relevant documents and displays their content and similarity scores. It demonstrates two important types of questions: single-document questions, where the answer can be obtained from one document, and multi-document questions, where information from two or more documents is required to construct the answer.